

Memoria de análisis y justificación del modelo.

1. Entidades, atributos y relaciones.

Para diseñar este modelo relacional, el enfoque principal ha sido cubrir fielmente las necesidades operativas del negocio y, sobre todo, garantizar la máxima transparencia y protección hacia el consumidor final. A nivel general, todas las entidades comparten atributos básicos de identificación (mediante IDs autonuméricos para asegurar relaciones sin fallos) y de trazabilidad temporal, como fechas de registro, de inicio o de fin. A continuación, se detalla la justificación de las tablas, sus atributos principales y sus relaciones estructurales.

1.1. Personal y ubicaciones.

El cliente nos solicita registrar la estructura física de la empresa y su plantilla para saber exactamente la actividad de cada trabajador. Por ello, se han definido las entidades de Sede y Empleado con sus datos de contacto básicos. Para organizar al equipo de manera eficaz sin complicar **demasiado** la base de datos, he propuesto añadir un atributo "cargo" que diferencie roles (como encargado, vendedor o personal de logística) y defina jerarquías de forma rápida y visual. Todo esto se articula con una relación de uno a muchos (1:N), ya que cada empleado está adscrito a una sola sede.

1.2. Clientes y direcciones.

Siguiendo los requisitos del negocio, un usuario debe poder almacenar varias direcciones de envío. He decidido separar las entidades de Cliente y Dirección en dos tablas distintas vinculadas (1:N). Esta decisión es fundamental para no duplicar datos personales sensibles (como el nombre, el email o la contraseña) cada vez que el cliente añade un nuevo destino. Adicionalmente, propongo incluir atributos logísticos específicos como el "bloque" y la "planta" en la dirección, algo vital para asegurar que los envíos son viables y evitar así costosas incidencias de reparto.

1.3. Catálogo, precios y transparencia.

El cliente pide estructurar el inventario de forma organizada (1:N), por lo que he establecido Categorías, Subcategorías y Productos, asegurando que un artículo no conviva duplicado en dos familias distintas. Por otro lado, y como propuesta propia para proteger al consumidor, he separado drásticamente el Histórico del Precio de Venta al Público (PVP) de la Promoción temporal y de su tabla puente Producto_Promoción. Separar el precio base del descuento aplicado es crítico para evitar prácticas engañosas, pues esto permite auditar por parte del cliente final que no se inflan los precios de forma artificial horas antes de aplicar una oferta.

1.4. Proveedores y logística de Inventario.

Para controlar quién suministra qué, la empresa necesita relacionar productos y proveedores. Esta relación de muchos a muchos (N:M) se resuelve con la tabla Acuerdo_Proveedor, cuyos atributos clave almacenan el precio de coste y el plazo de entrega, dejando un histórico auditable de las condiciones pactadas. Del mismo modo, el Stock y la Transferencia de Stock gestionan el control físico de unidades por cada sede,

registrando los movimientos internos autorizados por los empleados entre el almacén central y las distintas tiendas físicas.

1.5. Ventas y canal de distribución.

La venta online y la venta presencial deben gestionarse de forma totalmente separada (Pedido_Online, Envío, Ticket_Tienda y Devolución) debido a que su naturaleza logística y operativa es radicalmente distinta. Como propuesta técnica y ética innegociable, en las tablas de detalle (las líneas de pedido y de ticket) he forzado que se guarde el precio unitario. Congelar el precio en el instante exacto de la compra es una obligación contable que evita estafas: si el cliente tramita una devolución meses después, se le abonará exactamente lo que pagó, ignorando si el precio oficial del catálogo ha subido o bajado desde entonces.

1.6. Atención al cliente y sistema de fidelización.

Para la atención posventa, el Ticket_Incidencia centraliza las quejas o consultas almacenando su estado, asunto y nota de resolución. En el caso de la Valoración de los productos, se guarda la puntuación y se incluye un atributo de validación vital: la verificación de compra, indispensable para erradicar las reseñas falsas. Por último, el Movimiento_Puntos actúa como un libro contable estricto, registrando la cantidad exacta de saldo de fidelización ganado o gastado en cada transacción.

2. Preguntas de reflexión y obligatorias.

1. El texto menciona que un pedido online puede generar varios envíos. ¿Cómo lo has modelado y por qué?

Lo he modelado mediante dos relaciones separadas: de Pedido a Envío, y de Envío a Línea de Envío. De esta manera, si un pedido requiere artículos de distintos almacenes por falta de stock centralizado, generamos varias expediciones independientes. Cada envío podría tener su propio transportista y número de seguimiento en caso de estar conformado por varios bultos, lo que nos permite saber exactamente qué X producto viaja en Y bulto a la hora de localizar mercancía o reclamar extravíos.

2. Las ventas presenciales y los pedidos online son procesos distintos pero comparten productos y clientes. ¿Has optado por una sola tabla o dos tablas separadas? Justifica tu decisión.

He optado por dos tablas separadas. Si las juntásemos en una sola base usando un simple indicador de "tipo de venta", provocaríamos un caos de campos vacíos o nulos. Un ticket físico de tienda no necesita transportista ni dirección de entrega, y un pedido online no es cobrado por un vendedor en una caja registradora. Separarlas mantiene la información limpia, previene errores lógicos de programación y respeta la obligación fiscal de mantener series de facturación independientes.

3. El histórico de precios de un producto y el histórico de promociones son dos cosas distintas. ¿Cómo las diferencias en tu modelo?

Se encuentran en tablas completamente distintas. Una tabla custodia de forma inmutable el precio oficial (base) del producto a lo largo del tiempo, mientras que la otra registra las rebajas puntuales que aplican un porcentaje de descuento temporal. Mantener esta arquitectura demuestra con registros claros que la empresa no altera el precio base de forma engañosa justo antes de lanzar una campaña de ofertas, garantizando la transparencia frente al consumidor.

4. El saldo de puntos de fidelización "debe poder calcularse siempre desde el histórico de movimientos". ¿Qué implicación tiene esto en el diseño? ¿Guardarías también un campo de saldo_actual en la tabla de clientes?

La principal implicación es que no se debe guardar un campo de saldo fijo en la tabla del cliente. Almacenar un número estático supone un riesgo crítico de corrupción de datos; si hay un fallo de sincronización en el sistema, el usuario podría perder sus puntos arbitrariamente. Al calcular el saldo en tiempo real, sumando y restando su historial de movimientos completo, el resultado siempre será exacto, matemáticamente demostrable y seguro frente a errores.

5. Un cliente sin cuenta (compra presencial) puede luego vincularse a una cuenta online. ¿Cómo has previsto esto en tu modelo?

El modelo contempla que el identificador del cliente en la tabla de tickets de tienda pueda quedar en blanco (nulo), permitiendo registrar ventas anónimas respetando la privacidad. Si en el futuro ese usuario crea una cuenta y presenta su comprobante de papel, el sistema simplemente actualiza ese registro llenando el hueco con su nuevo ID de usuario, vinculando así su historial presencial al instante sin complicación.

6. ¿Hay algún requisito del texto que hayas decidido NO implementar o simplificar? Si es así, explica por qué y qué impacto tiene.

He decidido no implementar un sistema de control de acceso (permisos, contraseñas de red o roles de servidor) para los empleados. Aunque el texto mencionaba encargados y vendedores, el impacto de omitir una tabla de credenciales informáticas es razonable para la situación actual, pues aún no sabemos realmente cómo será la intranet ni el cómo aplicarán exactamente el control de acceso. Lo he simplificado usando un campo de texto enumerado ("cargo") en su propia ficha. Así identificamos las responsabilidades operativas de cada trabajador sin saturar la base de datos con tablas que no sabremos si necesitaremos rediseñar más adelante.